

Flutter Week 3

CRUD Operations & Local Data

A To-Do application demonstrating task creation, editing, deletion, completion, ListView.builder, Dart List, setState() and Provider.

Project	To-Do App / TaskFlow
Platform	Flutter / Android
IDE	Visual Studio Code
Architecture	Provider-based state management
Focus	CRUD Operations & Local Data

Submitted Project Report

1. Introduction

This Week 3 project was created to learn CRUD operations using local application data and to build a functional To-Do application. The implementation focuses on managing a list of tasks, updating task state, displaying tasks efficiently, and moving from direct `setState()` handling toward Provider-based state management.

2. Objective

- Create a To-Do App for managing daily tasks.
- Add a new task to the task list.
- Edit an existing task.
- Delete a task.
- Mark a task as completed.
- Display tasks using `ListView.builder`.
- Practice Dart List and CRUD operations.
- Use `setState()` initially and then change the state-management approach to Provider.

3. Tasks Completed

- Flutter To-Do project structure created and organized inside the lib folder.
- Task model and task provider files separated for maintainability.
- Home screen created to display and manage tasks.
- Task editor bottom sheet implemented for adding and editing tasks.
- Task tiles created for displaying individual tasks.
- Empty state widget added for a clean interface when no tasks are available.
- Provider used to manage task data and theme state across the application.

4. Topics Covered

Dart List	ListView.builder
CRUD Operations	setState()
Provider	ChangeNotifierProvider
Consumer	Task State Management
Task Model	Reusable Widgets

5. Project Structure

The project separates task management, screens, theme configuration and reusable widgets into individual Dart files inside the lib folder. This keeps the application organized and makes each feature easier to maintain.

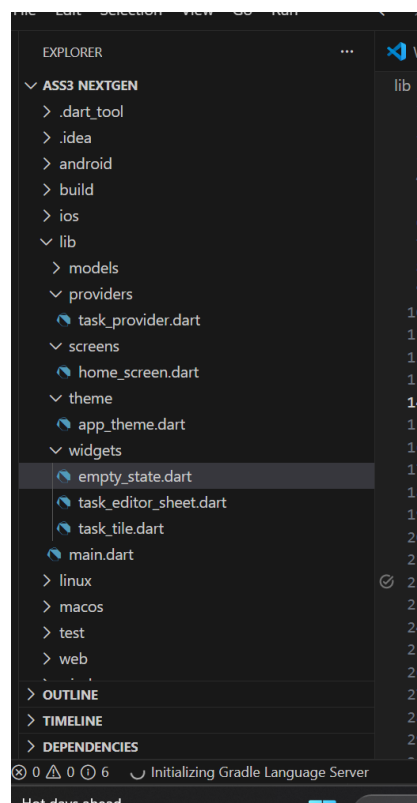


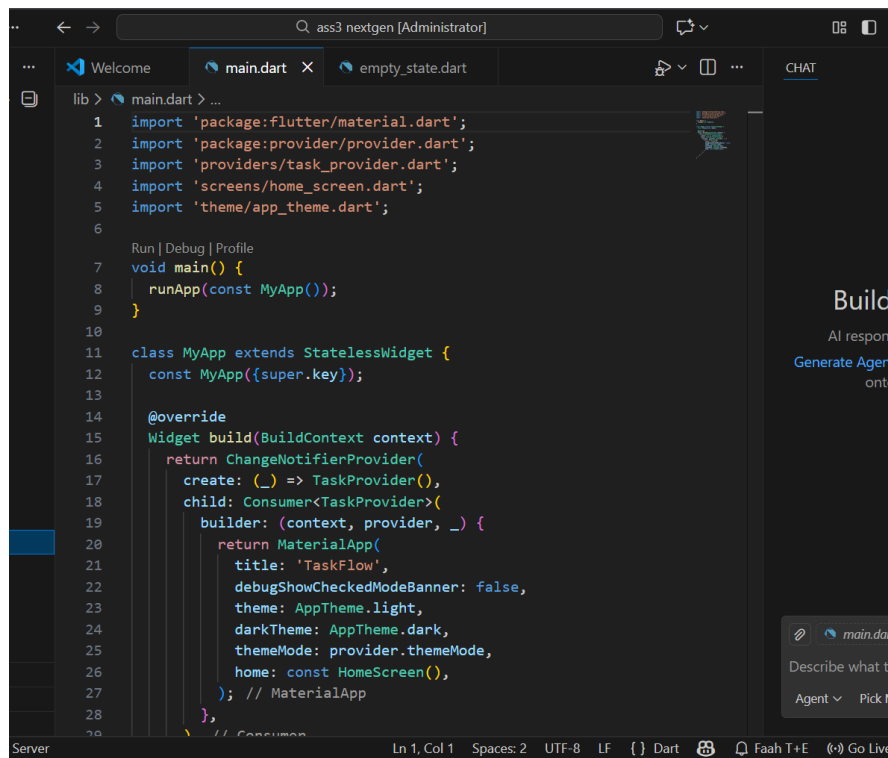
Figure 1 — Flutter project structure showing models, providers, screens, theme and reusable widgets.

6. Main Application

The main.dart file initializes the Flutter application and connects the task provider with the application using ChangeNotifierProvider and Consumer. The MaterialApp uses the centralized AppTheme and opens the HomeScreen.

7. Provider & State Management

The application uses Provider to manage task-related state. ChangeNotifierProvider creates the TaskProvider, while Consumer rebuilds the relevant UI when provider data changes. This approach replaces direct screen-level state handling with a more scalable state-management pattern.



```
lib > main.dart > ...
1 import 'package:flutter/material.dart';
2 import 'package:provider/provider.dart';
3 import 'providers/task_provider.dart';
4 import 'screens/home_screen.dart';
5 import 'theme/app_theme.dart';
6
7 void main() {
8   runApp(const MyApp());
9 }
10
11 class MyApp extends StatelessWidget {
12   const MyApp({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return ChangeNotifierProvider(
17       create: (_) => TaskProvider(),
18       child: Consumer<TaskProvider>(
19         builder: (context, provider, _) {
20           return MaterialApp(
21             title: 'TaskFlow',
22             debugShowCheckedModeBanner: false,
23             theme: AppTheme.light,
24             darkTheme: AppTheme.dark,
25             themeMode: provider.themeMode,
26             home: const HomeScreen(),
27           ); // MaterialApp
28         },
29       );
30   }
```

Figure 2 — main.dart showing Provider integration and MaterialApp configuration.

8. App Theme

The app_theme.dart file provides a central place for the application's visual identity. It defines reusable colors and light/dark ThemeData so the interface can remain consistent throughout the application.

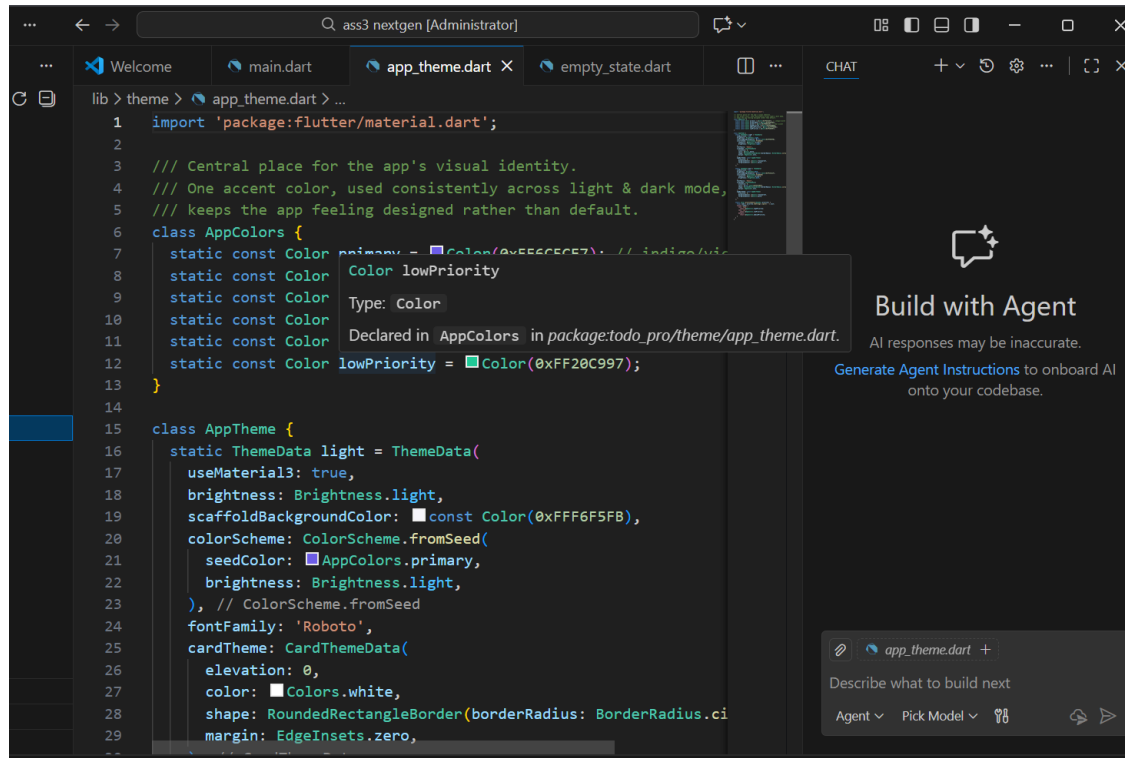


Figure 3 — app_theme.dart showing centralized colors and application theme configuration.

9. CRUD Operations

CRUD stands for Create, Read, Update and Delete. In this To-Do application, these operations form the main task-management cycle.

Operation	Application Function
Create	Add a new task using the task editor.
Read	Display available tasks in the task list.
Update	Edit task information and change completion status.
Delete	Remove an existing task from the list.

10. Add & Edit Task

The task editor bottom sheet provides the interface for entering task information. The same component can support both creating a new task and editing an existing task, reducing duplicated UI code.

11. Delete Task

Delete functionality removes the selected task from the managed task collection. The task list then rebuilds so the user immediately sees the updated data.

12. Mark Task as Completed

A task can be marked as completed from the task tile. The completion state updates the task data and the interface reflects the new status.

13. Display Tasks with `ListView.builder`

`ListView.builder` is used to display the task collection efficiently. It builds task tiles as needed and allows the list to update when the underlying provider state changes.

14. Empty State

When there are no tasks to display, the application uses an `EmptyState` widget instead of showing a blank screen. This gives the user clear visual feedback and a better experience.

15. setState() to Provider

The task demonstrates the progression from local setState() based state handling to Provider. setState() is useful for simple local UI changes, while Provider with ChangeNotifier provides a cleaner structure when multiple widgets depend on shared task data.

16. Application Workflow

Step	Flow
1	Open the To-Do Home Screen
2	View existing tasks using ListView.builder
3	Add a new task
4	Edit or update a task
5	Mark a task as completed
6	Delete a task when no longer needed
7	Provider updates the UI automatically

17. Running the Application

The project is designed to run as a Flutter application from Visual Studio Code on an Android emulator or other configured Flutter device. The implementation shown in the provided screenshots demonstrates the organized project files, provider setup and centralized theme configuration.

18. Conclusion

This Week 3 task provided practical experience with Dart Lists, ListView.builder and complete CRUD operations. It also demonstrated the transition from setState() to Provider for cleaner state management. The resulting To-Do application provides a strong foundation for adding more advanced local-data features in future Flutter tasks.